

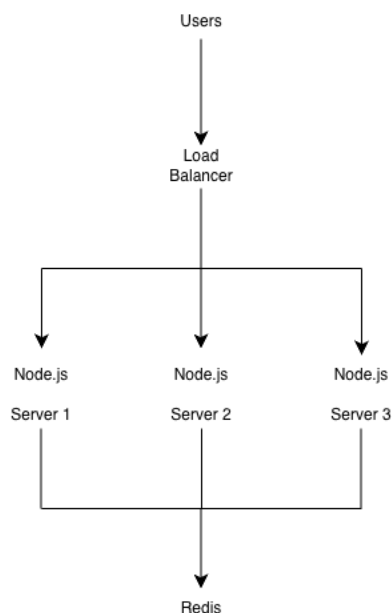
Improving Scalability and Load Balancing of a Containerized Cloud Application

Jayden Williams

Chapter 1 – Project Vision

The goal of this project is to build on the cloud application I created in my previous project. The original application uses a Node.js web server and Redis to keep track of the number of visitors to the website. In this project, I want to make the application able to handle more users by running multiple copies of the Node.js server instead of only one. A load balancer will sit between the users and the servers to decide which server should handle each request. All the servers will still use the same Redis database so they can share the visitor information. My goal is to see how adding multiple servers and a load balancer can make the application more scalable and allow it to handle more traffic.

10,000 Feet View Architecture:



The architecture starts with the users, who send requests to the application through the load balancer. The load balancer receives each request and decides which Node.js server should handle it. Instead of having only one Node.js server like in my previous project, there will be three servers that can share the workload. Each Node.js server will communicate with the same Redis database, which will store the visitor information. This setup allows the application to handle more users because the work can be spread across multiple servers instead of having everything handled by one server.

Chapter 2 – Project Proposal

To implement this project, I will take the Node.js and Redis application from my previous project and modify it to run multiple Node.js servers at the same time. I will use Docker to create and run the different application containers and set up a load balancer to send user requests to the available Node.js servers. I will keep Redis as the shared database so that all the servers can access the same visitor information. After setting up the system, I plan to test it by sending different amounts of traffic to the application and seeing how the servers handle the requests. I will look at things such as response time, how the requests are divided between the servers, and how the system performs when the amount of traffic increases. This will allow me to see if using multiple servers and a load balancer makes the application more capable of handling a larger number of users.